ZikShare

ACADEMIC VAULT

UNIZIK FRESHER EDITION | 2026

CSC 101: Introduction to Computer Science - Hardware, Data and Algorithms

What actually happens inside a computer, and how to think in algorithms

FACULTY PRIMER

Faculty Primer

Faculty of Physical Sciences
Computer Science | 100L

Written for students, by students.

Practical notes, worked examples, practice questions and a revision plan.

Free to download. No fee, no payment step, no watermark.

Independent study support | Not an official UNIZIK publication

START HERE

Why this pack exists

CSC 101 is often taught as a tour of parts, but the exam tests understanding. This pack explains the stored-program model, number systems, data representation, operating systems, networks and algorithmic thinking, with worked binary and flowchart exercises.

This is a student-written study companion, not a reproduced lecture handout or a leaked examination paper. Use it to organise the topics you are taught, practise the kind of thinking your course rewards, and walk into class with better questions.

What you will be able to do

- Explain the stored-program architecture and the roles of CPU, memory and storage.
- Convert between binary, decimal and hexadecimal and perform simple binary arithmetic.
- Represent text, images and sound in digital form and explain the trade-offs.
- Design a simple algorithm as pseudocode and a flowchart, and trace it for correctness.

How to use this pack

1. Skim the whole pack once in 20 minutes so you know what is inside.
2. Work through one section at a time with your lecture note and recommended text beside you.
3. Attempt the practice questions before reading the answers. Mark what you missed.
4. Use the final checklist to plan your revision week, one tick per study session.

> **Free to download, always**
> This pack is free. There is no fee, no subscription and no payment step before download, and the full PDF is not watermarked or locked. Share the link with a coursemate instead of forwarding the file.

> **Confirm your official course outline**
> Course codes, credit units and outlines can differ by department and session. Treat this pack as supplementary support and confirm the current outline with your lecturer, course rep or departmental handbook before an assessment.

The stored-program model

Modern computers follow the idea that instructions and data live in the same memory and are fetched and executed in sequence.

- CPU: control unit fetches and decodes, arithmetic logic unit executes.
- Registers: small, fast storage inside the CPU.
- Main memory: volatile, holds running programs and data.
- Secondary storage: persistent, slower, larger.
- Input and output devices move data between the machine and the world.

- The fetch-decode-execute cycle repeats for each instruction.

Number systems and data representation

Computers store patterns of bits. Everything else is a convention layered on top.

- Binary to decimal: sum each bit times its place value.
- Decimal to binary: divide repeatedly by 2 and read remainders upward.
- Hexadecimal groups four bits and is used for compact memory addresses and colours.
- One byte is eight bits; ASCII uses seven or eight bits per character, Unicode uses variable encoding.
- Images are stored as pixel values; resolution and colour depth trade quality against size.
- Sound is sampled at intervals; sampling rate and bit depth determine fidelity and file size.

> **Practice conversion by hand**
> Exams rarely allow calculators for number-system questions. Convert 156 to binary and back until it takes under a minute.

Algorithms, pseudocode and flowcharts

An algorithm is a finite, unambiguous sequence of steps that solves a problem.

- Sequence, selection and iteration are the three control structures.
- Pseudocode should be readable by a human and precise enough to implement.
- Flowchart symbols: oval for start and end, parallelogram for input/output, rectangle for process, diamond for decision.
- Trace a loop by writing a table of variable values for each iteration.
- Test with normal, boundary and invalid inputs.
- An algorithm that produces an answer but never terminates is not correct.

What a computer actually does

A computer is a machine that executes instructions. Its architecture comprises a processor, memory, input and output devices, and storage, connected by buses. The processor fetches an instruction, decodes it, executes it and writes the result back - the fetch-decode-execute cycle, repeated billions of times per second.

Programs and data share memory in the stored-program model, which is what makes general-purpose computing possible: the machine does not change, the instructions do. Memory hierarchy - registers, cache, RAM, disk - trades speed against capacity and cost, and understanding that hierarchy explains most performance behaviour.

Data is represented in binary because digital circuits distinguish two voltage levels reliably. Numbers, characters and images are all encoded: two's complement for signed integers, ASCII and Unicode for characters, pixel values for images. Knowing the encoding explains overflow, rounding error and why text files have encodings at all.

Algorithms and programming

An algorithm is a finite, unambiguous sequence of steps that solves a problem. The same problem admits many algorithms, and they differ in time and space complexity. Big-O notation describes how running time grows with input size, which matters more than constant factors once inputs are large.

Programs are built from sequence, selection and iteration, plus abstraction through functions and data structures. Choosing the right structure - list, stack, queue, dictionary, tree - often determines whether a solution is practical.

Debugging is a method, not a mood. Reproduce the fault, isolate it by narrowing the input or the code path, form a hypothesis, test it, and fix the cause rather than the symptom. Printing intermediate values or using a debugger is faster than guessing.

> **Trace it by hand**
> Run the algorithm on paper with a small input. Most logic errors become obvious within three iterations.

Systems, networks and security

An operating system manages processes, memory, files and devices, and enforces protection between users and programs. It schedules the processor, allocates memory, and handles input and output through drivers. Concurrency - multiple tasks appearing to run at once - introduces race conditions and requires synchronisation.

Networks move data in packets across layers, from physical transmission up through addressing and routing to applications. The layered model allows independent development: a change in wireless technology does not require rewriting a web browser.

Security rests on confidentiality, integrity and availability. Threats include malware, phishing, injection attacks and social engineering; defences include strong authentication, updates, least privilege, encryption and backups. Human factors are usually the weakest link, which is why security awareness training matters more than any single tool.

WORKED EXAMPLES

Do it once, slowly

These are not model answers to copy. They show the reasoning a strong student would use, step by step, so you can repeat the process on a question you have not seen.

1. Analysing a simple algorithm

Find the largest value in a list of n numbers. What is the cost?

1. Set the first element as the current maximum.
2. Compare each remaining element once.
3. Replace the maximum when a larger element is found.
4. Count operations: n - 1 comparisons, so time is linear, O(n).
5. Note that no comparison-based algorithm can do better, because every element must be examined.
6. Confirm space usage is constant, O(1).

> **What to carry forward**
> Count the dominant operation and express growth, not seconds.

2. Debugging a loop that never ends

A while loop that should count from 1 to 10 never terminates.

1. Read the loop condition and the update statement.
2. Check whether the update actually changes the variable in the condition.
3. Trace three iterations on paper.
4. Check for an off-by-one that prevents the exit condition being reached.
5. Add a temporary counter with a print statement to observe the variable.
6. Fix the update, then re-test with both a normal and an edge case.

> **What to carry forward**
> Trace the variable. Infinite loops are almost always a failure to change the thing you are testing.

EXAM TECHNIQUE

Questions with model answers

Written in the shape your course is usually examined, with the marking logic spelled out. Attempt each one under time before you read the outline.

Q1. Explain the fetch-decode-execute cycle.

10 marks

1. Fetch the instruction from the address in the program counter.
2. Decode the operation and operands.
3. Execute, using the ALU or memory access.
4. Store the result and update the program counter.

Q2. Compare two algorithms for the same problem using Big-O notation.

15 marks

1. Choose a problem, such as searching.
2. Describe linear search, O(n), and binary search, O(log n).
3. State the precondition for binary search: sorted data.
4. Explain why the difference matters at scale.
5. Note that constants matter for small inputs.

> **Where marks are lost**
> Claiming one algorithm is always faster. Preconditions and input size matter.

REFERENCE

Glossary

Use these definitions in your own sentences; a copied definition rarely earns full marks.

Algorithm. A finite, unambiguous sequence of steps solving a problem.

Big-O notation. A description of how an algorithm's cost grows with input size.

Binary. A base-2 number system used by digital circuits.

Two's complement. A scheme for representing signed integers in binary.

Compiler. A program that translates source code into machine code before execution.

Interpreter. A program that executes source code directly, statement by statement.

Operating system. Software managing hardware, processes, memory and files.

Data structure. An organised format for storing and accessing data.

Packet. A unit of data transmitted across a network.

Protocol. An agreed set of rules for communication between systems.

Encryption. Encoding data so that only authorised parties can read it.

Race condition. A fault arising when concurrent operations depend on timing.

Your four-week plan

Week 1 - Representation

Binary.

- Convert between binary, decimal and hex.
- Add and subtract in two's complement.
- Encode ten characters in ASCII.

Week 2 - Algorithms

Design.

- Write three algorithms in pseudocode.
- Trace each by hand.
- Estimate their complexity.

Week 3 - Programming

Implementation.

- Implement one algorithm in a real language.
- Test with edge cases.
- Debug one deliberate fault.

Week 4 - Systems

Context.

- Describe the memory hierarchy.
- Explain layering in networks.
- List five security practices.

Where to read more

Entry-level texts you can find in the university library or a departmental reading list. Confirm editions with your lecturer.

- Any introduction to computer science text covering representation, algorithms and systems.
- A programming textbook for the language used in your course.
- A short guide to computer security fundamentals.

PRACTICE, NOT PREDICTION

Check your understanding

These questions are written for this pack to test the ideas above. They are not past examination questions and do not predict what your lecturer will set. Answer first, then compare.

1. Convert 45 to binary.

Answer: 45 = 32 + 8 + 4 + 1 = 101101 in binary.

2. Why is hexadecimal used in computing?

Answer: Because each hex digit maps exactly to four binary bits, making long binary values shorter and easier to read.

3. List the three basic control structures.

Answer: Sequence, selection (decision) and iteration (loops).

4. Write pseudocode to find the largest of three numbers.

Answer: Read a, b, c. Set max = a. If b > max then max = b. If c > max then max = c. Print max.

5. What happens during the fetch-decode-execute cycle?

Answer: The CPU fetches the next instruction from memory, decodes what operation it specifies, executes it, and stores any result before repeating.

Your 7-step action plan

Tick each step only when you have done it. A tick you cannot defend is worse than an empty box.

- [ ] Convert ten numbers between binary, decimal and hexadecimal without a calculator.
- [ ] Draw and label the components of a computer system.
- [ ] Write pseudocode for three simple problems and trace each with sample inputs.
- [ ] Draw one flowchart using correct symbols for a decision loop.
- [ ] Explain the difference between RAM and secondary storage in two sentences.
- [ ] Learn ten computing terms and define each without jargon.
- [ ] Practise one CSC 101 objective paper and review every error.

Keep going

One pack will not carry a semester. Pair this with your lecture notes, tutorials, and the past questions your department actually releases. If a topic still feels unclear, take a specific question to your lecturer or course rep - "I do not understand the difference between X and Y" gets a better answer than "I do not understand this course".

> **More from the ZikShare Academic Vault**
> Find faculty primers, study-skills guides, money and career packs, and digital-skills workbooks in the ZikShare Library. New packs are added every semester.

ZikShare Academic Vault | Faculty Primer | 100L | 2026 Fresher Edition

YOUR WORKSPACE

Notes, questions and things to ask

Use this page in class or during revision. Write questions you could not answer instead of copying slides.

Three ideas I can now explain without notes:

A question I still cannot answer:

For my lecturer or course rep:

Next revision action and date:

CSC 101: Introduction to Computer Science - Hardware, Data and Algorithms